Parallel Acceleration and Performance Analysis of Monte Carlo Simulation for Gold Investment Risk Assessment Using OpenMP and MPI

Andhika Narawangsa Susilo
Institut Teknologi Bandung
Bandung, Indonesia
13222036@mahasiswa.itb.ac.id

Jovan Hosea H. N.
Institut Teknologi Bandung
Bandung, Indonesia
13622005@mahasiswa.itb.ac.id

Kayla Pramudio B. A.
Institut Teknologi Bandung
Bandung, Indonesia
13222117@mahasiswa.itb.ac.id

***Abstract* – Monte Carlo simulation for financial risk analysis is computationally intensive because each stochastic price path requires multiple time-step updates. This study implements and compares OpenMP and MPI parallelizations of a 252-step Geometric Brownian Motion Monte Carlo model for gold investment risk assessment. The C++ implementations were evaluated on an Intel Core i5-12500H processor using 1, 2, 4, 8, 12, and 16 workers across workloads of 100,000, 1,000,000, and 10,000,000 simulation paths. For the primary 1,000,000-path workload, OpenMP achieved 3.18 s and 8.88× speedup using 16 threads, while MPI achieved 3.30 s and 8.60× speedup using 16 ranks. At 100,000 paths, MPI showed lower efficiency at high worker counts due to synchronization and aggregation overhead. At 10,000,000 paths, the overhead was amortized, and both implementations achieved comparable 16-worker performance: 31.777 s and 8.92× speedup for OpenMP, and 32.044 s and 8.85× speedup for MPI. Both implementations produced terminal-price estimates with relative errors below 0.04% from the analytical GBM expectation and generated probability-of-loss, VaR, and CVaR estimates.**

Keywords—High-Performance Computing, OpenMP, MPI, Monte Carlo, Geometric Brownian Motion, Value at Risk.

I. Introduction

The primary challenge in financial quantitative computing, particularly in gold investment, lies in the inability of simple deterministic analytical formulas to comprehensively map risk distributions for real world assets characterized by stochastic dynamics and continuous market uncertainty. Calculating expected final prices alongside extreme tail-risk indicators, such as Value at Risk (VaR) for a one year horizon, cannot be sufficiently characterized by a single deterministic estimate alone.

This operational reality forces analysts to generate millions to billions of potential future market paths via Monte Carlo path generation. The computational load escalates dramatically when attempting high resolution asset pricing across a massive number of simulation paths. Processing it at a multimillion or billion iteration scale transforms the problem into a critical performance bottleneck that demands a high performance computing infrastructure to deliver rapid and actionable risk insights. The workload consists of 1,000,000 simulation paths, each discretized into 252 trading-day steps.

This study compares two parallel programming models for Monte Carlo-based gold investment risk analysis: OpenMP and MPI. OpenMP distributes independent simulation paths among multiple threads within a shared-memory environment, whereas MPI distributes simulation paths among multiple ranks with separate memory spaces. The implementations are evaluated using 1,000,000 simulation paths and 252 time steps per path on a local multicore processor.

II. Baseline and Existing Computational Approaches

The standard computational baseline currently deployed by financial analysts to address stochastic probability problems relies on executing sequential Monte Carlo simulation. High level interpreted languages such as Python (leveraging Pandas or NumPy frameworks), the R language, or scripting macros inside Microsoft Excel are heavily utilized.

This conventional approach threads simulation iterations sequentially within a single threaded execution loop on a single CPU core. While highly straightforward to implement programmatically, this method suffers from severe scalability limits and prohibitive execution times. When confronted with institutional-grade computational requirements, such as evaluating millions independent simulation trajectories, the sequential approach under interpreted runtimes frequently triggers memory exhaustion errors or demands hours of runtime, rendering it obsolete for real-time strategic risk management and dynamic decision-making.

III. Parallel Computational Design

A. Parallelization Startegy

The Monte Carlo simulation is parallelized by exploiting the independence of individual simulation paths. Each path starts from the same calibrated initial parameters but uses an independent random-number sequence to generate a different gold-price trajectory. Therefore, the total number of simulation paths can be partitioned among multiple computational workers without requiring communication during the path-generation stage.

For a total of N simulation paths and p workers, each worker is assigned approximately N/p paths. Every worker independently performs the 252-step Geometric Brownian Motion calculation for its assigned paths and accumulates local statistical results, including the terminal-price sum, squared-price sum, loss count, drawdown, minimum price, maximum price, and terminal-price histogram.

After all workers finish their assigned paths, the local statistics are aggregated into global results. The final aggregated histogram is then used to estimate percentile

values, Value at Risk, and Conditional Value at Risk. This approach minimizes communication during the computationally intensive simulation phase and performs synchronization only during the final aggregation stage.

B. Unified Parallel Computational Architecture

The proposed computational architecture consists of four main stages: parameter initialization, workload partitioning, parallel path simulation, and result aggregation. During parameter initialization, the system reads the historical gold-price and risk-free-rate datasets to determine the initial price, risk-free rate, and annualized volatility. These parameters are shared or distributed to all computational workers before the simulation begins.

In the workload partitioning stage, the total Monte Carlo paths are divided among the available workers. Each worker initializes an independent random-number generator and simulates its assigned price paths using the 252-step Geometric Brownian Motion model. To avoid race conditions and reduce synchronization overhead, each worker stores terminal-price frequencies in a local histogram and maintains local statistical accumulators.

After the parallel simulation stage is completed, the local histograms and statistical accumulators are combined into a global distribution. The global results are then used to calculate the mean terminal price, standard deviation, probability of loss, drawdown, percentile values, Value at Risk, and Conditional Value at Risk.

The same logical architecture is implemented using two parallel programming models. In OpenMP, workers are implemented as threads within a shared-memory environment. In MPI, workers are implemented as ranks with separate memory spaces, where the final aggregation is performed through MPI communication primitives.

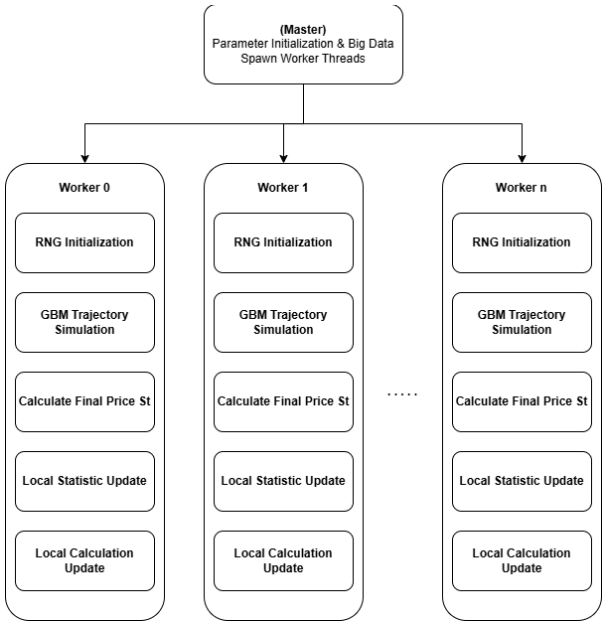

Fig 1. Parallel Worker Architecture

C. Computational Workflow

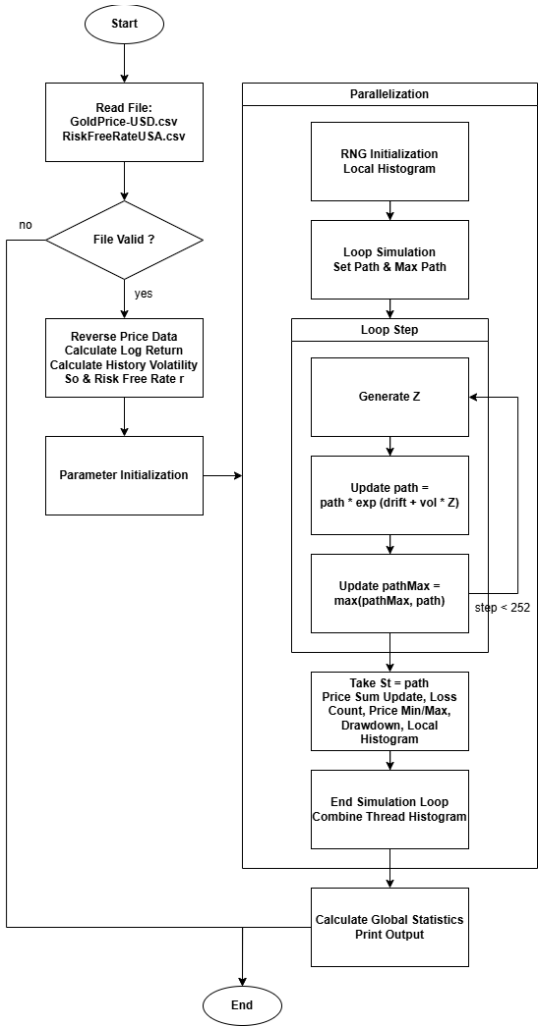

Fig 2. Monte Carlo Simulation Flowchart Diagram

The workflow is identical for both OpenMP and MPI during the Monte Carlo computation stage. The difference lies only in the execution mechanism: OpenMP assigns paths to threads through a shared-memory parallel loop, whereas MPI assigns paths to ranks and aggregates local results using message-passing operations.

- Input Phase: The system extracts raw historical parameters from the GoldPrice-USD.csv and RiskFreeRateUSA.csv datasets.
- Pre-Computation Phase: The orchestrator calculates the annualized historical volatility and the corresponding real-world risk-free rate.
- Parallel Execution Phase: Calibrated baseline variables trigger the parallel simulation loop. Each micro-thread autonomously evaluates the terminal exponential price step driven by independent standard normal random samples.
- Aggregation Phase: Frequency bin arrays are accumulated globally into the memory workspace of Rank 0 to reconstruct the terminal probability distribution.

IV. IMPLEMENTATION

The Monte Carlo risk-analysis system was implemented in ISO C++17 using two functionally equivalent parallel versions: OpenMP and MPI. The OpenMP implementation uses shared-memory parallelism, where simulation paths are distributed among multiple threads within a single process. The MPI implementation uses distributed-memory parallelism, where simulation paths are distributed among multiple ranks. Both implementations were compiled using optimization-enabled GCC-based toolchains with the -O3 optimization flag.

The input module reads historical gold-price and risk-free-rate data from semicolon-separated CSV files. Comma characters contained in numerical values are removed before the data are converted into floating-point values. The historical gold-price data are then used to calculate annualized volatility from daily logarithmic returns, while the latest gold price and risk-free rate are used as the initial GBM parameters.

Each Monte Carlo path represents one year of gold-price movement divided into 252 trading-day intervals. Therefore, the simulation applies the discretized Geometric Brownian Motion update iteratively for each time step:

$$S_T = S_0 \exp(\left(r - \frac{\sigma^2}{2}\right)\Delta t + \sigma\sqrt{\Delta t}Z)$$

where (St) is the gold price at time (t), (r) is the risk-free rate used as the drift parameter, (sigma) is the annualized historical volatility, (Zt) is a standard normal random variable, and (Delta t = 1/252). Each simulation path is updated 252 times before its final terminal price is obtained.

Each computational worker initializes an independent 64-bit Mersenne Twister random-number generator, {std::mt19937_64}, with a worker-specific seed. In the OpenMP implementation, each thread maintains its own random-number generator and local histogram. In the MPI implementation, each rank maintains its own random-number generator and local histogram. This design avoids race conditions during random-number generation and histogram updates.

To reduce memory usage, the implementation does not store all terminal prices individually. Instead, each worker maps its terminal-price results into a fixed 10,000-bin local histogram. After the simulation stage is completed, the local histograms and statistical accumulators are combined into global results. In OpenMP, this aggregation is performed after the parallel region finishes, whereas in MPI, the local results are combined using reduction operations. The global histogram is then used to estimate percentile values, Value at Risk, and Conditional Value at Risk. Because the histogram uses fixed bins, the resulting percentile-based risk metrics are treated as efficient approximations of the terminal-price distribution.

V. EXPERIMENTAL RESULTS AND ANALYSIS

1. Experimental Configuration

Table I. Experimental Configuration

Parameter	Configuration
Processor	Intel Core i5-12500H, 12th Generation
Physical cores	12 cores
Logical processors	16 logical processors
Base clock	2.50 GHz
Parallel models	OpenMP and MPI
Worker configurations	1, 2, 4, 8, 12, and 16
Simulation paths	1,000,000
Time steps per path	252
Historical gold-price records	2,189 observations
Initial price (S0)	$4,482.32
Risk-free rate (r)	4.47%
Historical volatility	17.63%
Programming language	C++
OpenMP compilation	g++ -O3 -std=c++17 -fopenmp
MPI compilation	mpicxx -O3 -std=c++17
Operating system	Windows 11 Home Single Language 25H2
Memory	16GB

2. Benchmark Timing Scope

To ensure a consistent interpretation of the benchmark results, the reported execution time was defined as the elapsed time of the parallel execution phase. Table II summarizes the components included and excluded from the measured execution time for the OpenMP and MPI implementations. The measurement includes the major parallel computation and synchronization costs, while file reading, GBM parameter calibration, final risk-metric calculation, and terminal output are excluded.

Table II. Benchmark Timing Scope

Component	OpenMP	MPI
MPI runtime initialization (MPI_Init)	–	Included
Thread creation / entry into parallel region	Included	–
Workload partitioning	Included	Included
Worker-specific RNG seed initialization	Included	Included
Initial synchronization	–	Included
Monte Carlo path simulation	Included	Included
Random-number generation	Included	Included
Local statistics and histogram updates	Included	Included
Thread/process synchronization	Included	Included
Statistical reduction	Included	Included
Histogram aggregation	Included	Included

MPI inter-process communication	–	Included
CSV file reading and GBM calibration	Excluded	Excluded
Final VaR/CVaR calculation and output printing	Excluded	Excluded
OS-level process creation by mpiexec before main()	–	Excluded

3. Computational Performance Comparison Analysis

The performance benchmark was conducted using 1,000,000 Monte Carlo simulation paths, with each path simulated over 252 Geometric Brownian Motion time steps. Both OpenMP and MPI implementations were evaluated using 1, 2, 4, 8, 12, and 16 workers under the same hardware and input-data conditions. The execution time, speedup, and parallel efficiency for each configuration are summarized in Table II.

The speedup and parallel efficiency calculated for each parallel framework is calculated using:

$$S_p = \frac{T_s}{T_p} \text{ and } E_p = \frac{S_p}{p} x\ 100\%$$

With:

$$S_p : Speedup\ using\ p\ workers$$
$$T_s : Sequential\ execution\ time$$
$$T_p : Execution\ Time\ with\ p\ workers$$
$$E_p : Effeiciency\ of\ p\ workers$$
$$p : workers\ used$$

Table III. Experimental Results on 1,000,000 Workload

Workers	Time		Speedup		Efficiency	
	OpenMP	MPI	OpenMP	MPI	OpenMP	MPI
1	28.22	28.36	1.00	1.00	100.00%	100.00%
2	13.45	13.43	2.10	2.11	104.87%	105.60%
4	6.78	7.40	4.16	3.84	104.01%	95.88%
8	4.88	5.44	5.78	5.21	72.30%	65.14%
12	3.68	3.52	7.66	8.05	63.86%	67.10%
16	3.18	3.30	8.88	8.60	55.47%	53.76%

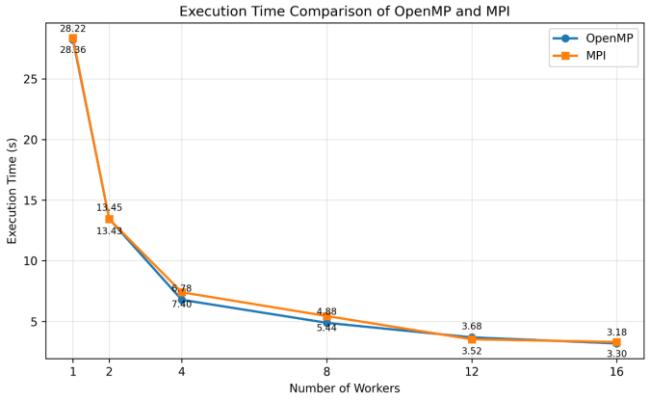

Fig 3. Execution Time vs Number of Workers

Fig 4. Speedup vs Number of Workers

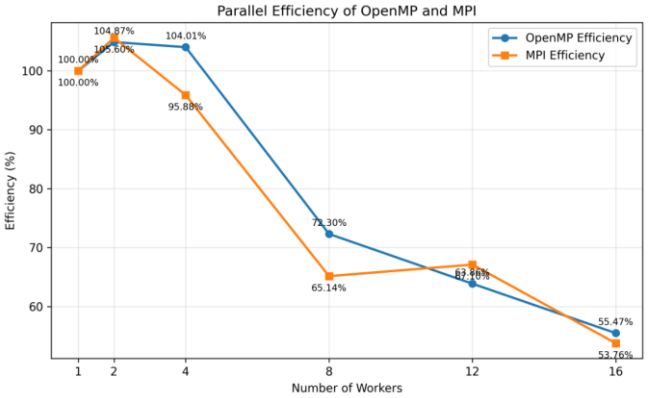

Fig 5. Parallel Efficiency vs Number of Workers

The OpenMP implementation reduced the execution time from 28.22 s using one thread to 3.18 s using 16 threads, corresponding to a maximum speedup of 8.88×. Similarly, the MPI implementation reduced the execution time from 28.36 s using one rank to 3.30 s using 16 ranks, achieving a maximum speedup of 8.60×. Both implementations demonstrate that Monte Carlo simulation is highly suitable for parallel execution because each simulation path can be calculated independently before the final aggregation stage.

As shown in Fig. 4 and Fig. 5, both OpenMP and MPI exhibit near-linear scaling at low worker counts. OpenMP achieved speedups of 2.10× and 4.16× using two and four threads, respectively, while MPI achieved speedups of 2.11× and 3.84× using two and four ranks. The slightly above-ideal efficiency values at low worker counts are likely caused by cache effects, CPU turbo boost behavior, and runtime measurement variation rather than a true superlinear parallel performance improvement.

The performance gain becomes less linear at higher worker counts. At 16 workers, OpenMP achieved the lowest execution time of 3.18 s, while MPI required 3.30 s. The parallel efficiency decreased to 55.47% for OpenMP and 53.76% for MPI at 16 workers, as illustrated in Fig. 6. This reduction is caused by synchronization overhead, memory contention, operating-system scheduling, and increased competition for processor resources. In addition, the evaluated processor provides 12 physical cores and 16 logical processors; therefore, the additional workers beyond 12 primarily utilize logical processors rather than fully independent physical cores.

Overall, both OpenMP and MPI significantly accelerated the Monte Carlo simulation workload. OpenMP produced the best execution time on the evaluated local multicore system, while MPI showed comparable performance and provides a distributed-memory model that can be extended to multi-node computing environments. These results confirm that parallel programming is effective for accelerating large-scale stochastic simulations with a high number of independent computation paths.

4. Workload-Size Scalability Analysis

To evaluate the influence of computational workload on parallel scalability, additional experiments were conducted using 100,000 and 10,000,000 simulation paths, while the 1,000,000-path experiment was retained as the primary benchmark.

Table IV. Experimental Results on 100,000 Workload

Workers	Time		Speedup		Efficiency	
	OpenMP	MPI	OpenMP	MPI	OpenMP	MPI
1	2.809	2.834	1.00	1.00	100.00%	100.00%
2	1.374	1.357	2.04	2.09	102.22%	104.42%
4	0.679	0.720	4.14	3.94	103.42%	98.40%
8	0.515	0.565	5.45	5.02	68.18%	62.70%
12	0.404	0.425	6.95	6.67	57.94%	55.57%
16	0.334	0.394	8.41	7.19	52.56%	44.96%

Table V. Experimental Results on 10,000,000 Workload

Workers	Time		Speedup		Efficiency	
	OpenMP	MPI	OpenMP	MPI	OpenMP	MPI
1	283.516	283.519	1.00	1.00	100.00%	100.00%
2	134.355	134.485	2.11	2.11	105.51%	105.41%
4	67.423	67.393	4.21	4.21	105.13%	105.17%
8	47.676	48.257	5.95	5.88	74.33%	73.44%
12	35.419	36.038	8.00	7.87	66.71%	65.56%
16	31.777	32.044	8.92	8.85	55.76%	55.30%

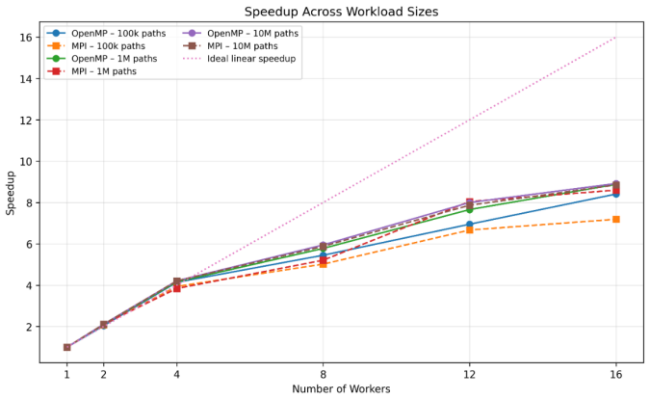

Fig 6. Speedup Across Workload Sizes

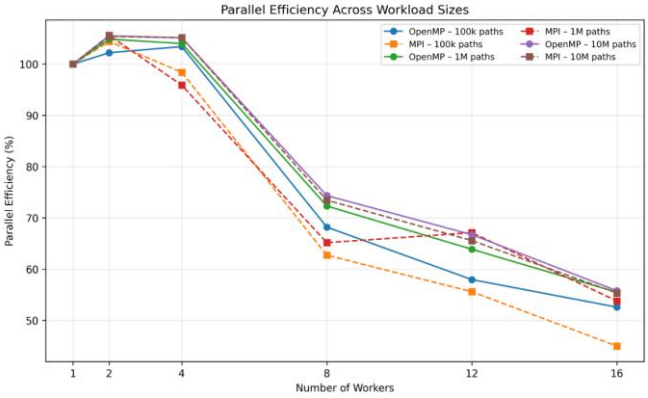

Fig 7. Parallel Efficiency Across Workload Sizes

Table IV & V shows that execution time increases proportionally with workload size for both OpenMP and MPI. At 100,000 paths, OpenMP achieved 0.334 s and MPI achieved 0.394 s using 16 workers. The performance gap is more visible at this smaller workload because the fixed costs of worker management, synchronization, and final aggregation represent a larger fraction of total runtime. At 1,000,000 and 10,000,000 paths, the execution times of OpenMP and MPI become increasingly similar because the computational cost of Monte Carlo path generation dominates the relative communication overhead.

As shown in Fig. 6, both implementations obtain near-linear speedup up to four workers for all workload sizes. At higher worker counts, the speedup curve becomes flatter. This effect is particularly visible for MPI at the 100,000-path workload, where the speedup increases only from 6.67× at 12 ranks to 7.19× at 16 ranks, compared with OpenMP increasing from 6.95× to 8.41×. This indicates that MPI synchronization and reduction costs are more noticeable when each rank receives a relatively small number of simulation paths.

Fig. 7 further confirms the reduction in parallel efficiency beyond four workers. For the 100,000-path workload, MPI efficiency decreases to 44.96% at 16 ranks, while OpenMP retains 52.56% efficiency. However, at 10,000,000 paths, both models produce nearly identical results at 16 workers, reaching 8.92× speedup for OpenMP and 8.85× for MPI, with efficiencies of 55.76% and 55.30%, respectively. Therefore, the MPI overhead is amortized as the workload becomes larger, because each worker performs substantially more independent GBM calculations before synchronization and aggregation are required.

Overall, the results indicate that Monte Carlo simulation is highly scalable for workloads in the order of millions of paths. Small workloads remain more sensitive to parallel overhead, especially for MPI, whereas large workloads provide sufficient computation per worker to maintain comparable OpenMP and MPI scalability.

5. Numerical Verification and Risk Metrics

a) Numerical Verification

The numerical validity of the simulation was evaluated by comparing the simulated mean terminal price with the analytical expectation of the Geometric Brownian Motion model:

$$E[S_T] = S_0 e^{(rT)}$$

Using the calibrated parameters, the analytical expected terminal price was $4687.23. The fastest OpenMP configuration, using 16 threads, produced a simulated mean terminal price of $4687.61, corresponding to a relative error of approximately 0.008%. The fastest MPI configuration, using 16 ranks, produced a simulated mean terminal price of $4685.39, corresponding to a relative error of approximately 0.039%.

These small deviations indicate that both parallel implementations preserve the expected statistical behavior of the GBM model. The remaining difference from the analytical value is expected because Monte Carlo simulation uses finite random samples and different worker configurations generate independent random-number streams.

b) Risk Metric Estimation

The terminal-price distribution was used to estimate the probability of loss, Value at Risk, and Conditional Value at Risk. The probability of loss is defined as the proportion of simulation paths with a terminal price below the initial gold price (S_0). VaR represents the estimated loss threshold at a specified confidence level, while CVaR represents the average loss within the worst tail region beyond the VaR threshold.

Table VI summarizes the numerical verification and risk metrics obtained from the fastest OpenMP and MPI configurations. Both implementations produced closely comparable risk estimates, with small differences caused by stochastic sampling variation and the use of independent random-number sequences.

Table VI. Numerical Verification and Risk Metrics of the Parallel Configurations

Metric	OpenMP (16 Threads)	MPI (16 Ranks)
Execution Time (s)	3.18	3.297
Simulated Mean Price	$4,687.61	$4,685.39
Analytical Mean Price	$4,687.23	$4,687.23
Relative Error	0.01%	0.04%
Final Price Std Dev	$832.44	$832.68
Probability of Loss	43.24%	43.52%
Average Drawdown	10.91%	10.93%
VaR 95%	$1,029.08	$1,030.57
VaR 99%	$1,419.46	$1,422.44
CVaR 95%	$1,267.71	$1,269.74

The results indicate that the estimated probability of loss is approximately 43%, meaning that a substantial portion of the simulated one-year scenarios end below the initial gold price. At the 95% confidence level, the estimated VaR is approximately $1030 per unit of gold-price exposure, while the CVaR indicates that the average loss within the worst 5% of scenarios is approximately $1268–$1270. Because the percentile values are derived from a fixed-bin histogram, the reported VaR and CVaR values should be interpreted as efficient approximations of the simulated terminal-price distribution.

The source code and the result for the simulation is available here

VI. Conclusion

This study evaluated OpenMP and MPI implementations of a 252-step GBM Monte Carlo simulation for gold investment risk assessment using 100,000, 1,000,000, and 10,000,000 simulation paths. The 1,000,000-path workload was used as the primary benchmark. At 16 workers, OpenMP achieved 3.18 s with an 8.88× speedup, while MPI achieved 3.30 s with an 8.60× speedup.

The workload-size analysis showed that parallel overhead is more significant for smaller workloads. At 100,000 paths, MPI efficiency decreased to 44.96% at 16 ranks, compared with 52.56% for OpenMP, due to process synchronization and reduction costs. As the workload increased, these fixed overheads were amortized. At 10,000,000 paths, OpenMP and MPI achieved nearly identical performance, reaching 8.92× and 8.85× speedup, respectively.

Both implementations preserved the expected statistical behavior of the GBM model, with mean terminal-price errors below 0.04% relative to the analytical expectation. Therefore, OpenMP provides the lowest runtime for the evaluated single-node multicore system, while MPI remains competitive for sufficiently large workloads and is suitable for extension to distributed multi-node execution.

References

[1] N. A. Zakaria, M. F. Ahmad, and M. S. M. Kasihmuddin, "Modeling and forecasting gold price using Geometric Brownian Motion," AIP Conf. Proc., vol. 2138, no. 1, pp. 030018-1–030018-6, Aug. 2019.

[2] S. Maruddani and A. Prahutama, "Value at Risk portfolio using Monte Carlo simulation," J. Phys.: Conf. Ser., vol. 855, no. 1, pp. 012027-1–012027-7, May 2017.

[3] V. Alexandrov, "Parallel Monte Carlo algorithms for financial derivatives pricing," Math. Comput. Simul., vol. 47, no. 2, pp. 113–122, Aug. 1998.

[4] M. B. Giles, "Multi-level Monte Carlo path simulation," Oper. Res., vol. 56, no. 3, pp. 607–617, Jun. 2008.

[5] T. J. Linsmeier and N. D. Pearson, "Value at Risk," Financ. Anal. J., vol. 56, no. 2, pp. 47–67, Mar. 2000.